INDEPENDENT SECURITY RESEARCH

Watching the Internet Knock

A 24-hour SSH and RDP honeypot on a bare public cloud VM. What automated attackers actually try, how fast they find you, and what it reveals about the background noise every exposed server on the internet lives in.

Author
Tomer Shapira

August 2026

12,038
Connection attempts logged in 24 hours

17 sec
Time from deployment to first RDP probe

225
Unique source IPs, 51 countries

DECOY SERVICES: NO REAL BACKEND, NO PATH TO A REAL SHELL

GCP e2-small · Debian 12

github.com/tomershapira103/Honeypot

Why point a decoy at the open internet?

Any server with a public IP starts receiving unsolicited connection attempts almost immediately, long before a human ever finds it. That traffic is usually invisible: it lives in auth logs nobody reads, on production boxes where "someone tried to log in and failed" isn't interesting enough to investigate. This project made that traffic the entire point.

I stood up a small cloud VM running two protocol-level decoys: nothing behind them, no real filesystem, no real remote desktop, no path to an actual shell, and let the internet find it on its own. Every connection, every credential guess, and every client fingerprint was logged and analyzed to answer four concrete questions:

1. **How long until first contact?**
   From the moment the VM goes live, how many seconds or minutes pass before the first scanner finds each open port?
2. **How much traffic, sustained over time?**
   Is this a one-time sweep or a continuous, ongoing background hum of scanning?
3. **What credentials do attackers actually try?**
   Which usernames and passwords dominate real-world brute-force dictionaries in 2026?
4. **What's doing the knocking?**
   Generic botnet tooling, purpose-built scanners, or something closer to a human with a terminal?

> **WHY THIS MATTERS**
>
> Every one of these connection attempts is aimed at the same numbered ports on *every* internet-facing box, all the time. Seeing the shape of that traffic (its speed, its volume, its credential lists) is directly useful for anyone hardening a server, sizing an intrusion-detection baseline, or just deciding whether "nobody would bother attacking my tiny VM" is a safe assumption. It isn't.

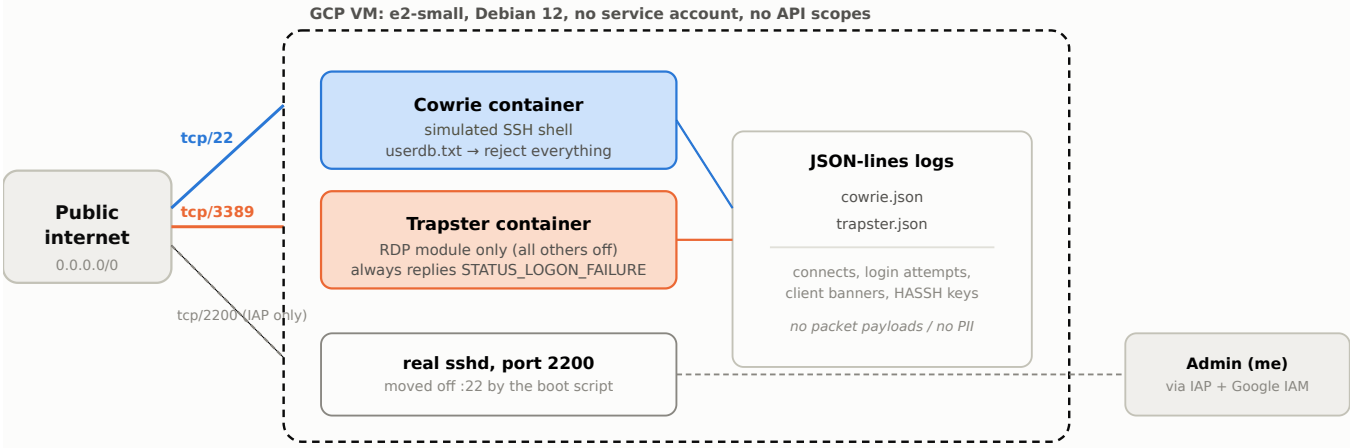
Ground rules for the study

- **Protocol-level decoys only.** Cowrie presents a simulated Linux shell over SSH; Trapster completes just enough of the RDP handshake to capture a username. Neither has a real backend: an attacker can never get further than a login prompt.
- **Deny-all authentication.** Every credential is rejected, on both services, always. This maximizes the number of distinct guesses captured, at the cost of never observing post-login behavior (see Limitations, p.9).

- **Admin access preserved separately.** Real SSH access is moved to a non-standard port, reachable only through an authenticated cloud tunnel, never exposed to the ports under study.

How the decoy was built

The setup runs on a single small VM in Google Cloud, provisioned by an idempotent shell script that creates an isolated VPC, opens exactly the ports under study, and deploys two containerized honeypots.



Every resource-creation step in the provisioning script is idempotent, so a partial run can simply be re-run. Analysis is a separate, offline step run against the downloaded logs.

Design choices worth calling out

- **Real admin access never touches the studied ports.** SSH for administration lives on port 2200, reachable only through Google's Identity-Aware Proxy, authenticated by IAM identity rather than source IP, so it keeps working from anywhere with nothing to allowlist.
- **Blast radius is capped at the VM.** The instance runs with `--no-service-account --no-scopes`, so even a hypothetical full compromise of the box has no credentials to reach any other cloud resource.
- **RDP passwords are never actually plaintext.** RDP's NLA/CredSSP handshake encrypts the password before it's sent. Trapster captures a plaintext *username* pre-TLS, plus (if the client completes NLA) a **NetNTLMv2 hash**: offline-crackable, but not a readable password on its own. The analysis keeps these labeled and separate from SSH's genuinely plaintext credentials.

Deployed	**2026-08-16 19:34:45 UTC**
Observation window	**24 hours**
SSH honeypot	**Cowrie (medium-interaction)**
RDP honeypot	**Trapster Community (RDP module)**
VM	**GCP e2-small, Debian 12**
Analysis	**analysis/analyze_logs.py (Python)**

The internet found it before I'd finished deploying

The most immediate finding wasn't a credential or a client fingerprint. It was the clock. Mass scanning of the public IPv4 space is fast enough that "newly provisioned" and "already being probed" are nearly the same moment.

FIRST SSH (:22) PROBE

5m 46s

after the VM went live

FIRST RDP (:3389) PROBE

17 seconds

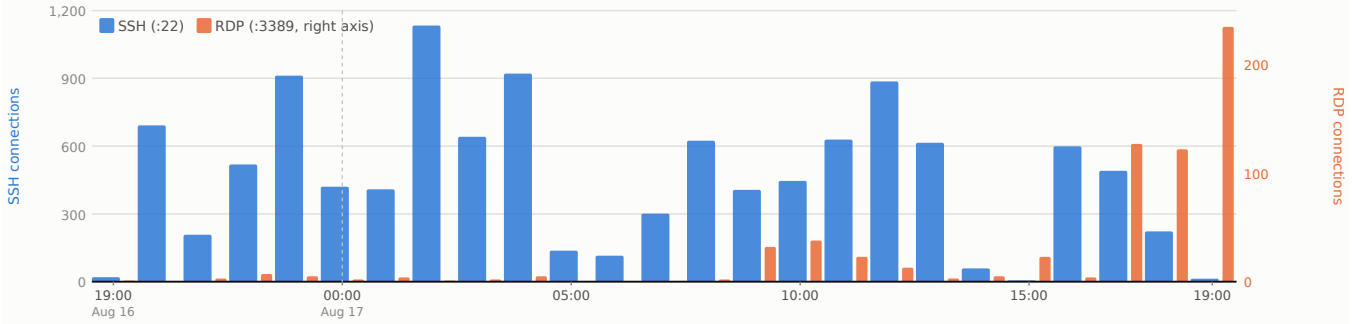
after the VM went live

BOTH EFFECTIVELY INSTANT

RDP was found in 17 seconds, SSH in under six minutes. Neither number leaves room for "nobody scans small VMs": commercial and research internet-scanning platforms sweep the full public IPv4 address space on a cycle measured in minutes to hours, so a freshly provisioned address gets discovered before most manual setup checklists are even finished.

Attack volume per hour over the 24-hour window

SSH connections (left axis) and RDP connections (right axis), binned hourly from deployment at 19:34 UTC. SSH total: **11,426** · RDP total: **612** · SSH ran about **19×** hotter than RDP throughout.



Traffic is concentrated, not evenly spread

On SSH, the busiest **10 of 173** unique source IPs (6% of sources) account for **53%** of all traffic. Averaged across all sources, each SSH IP made about 66 attempts: this isn't 173 people trying once each, it's a smaller number of persistent, repeat-hitting scanners.

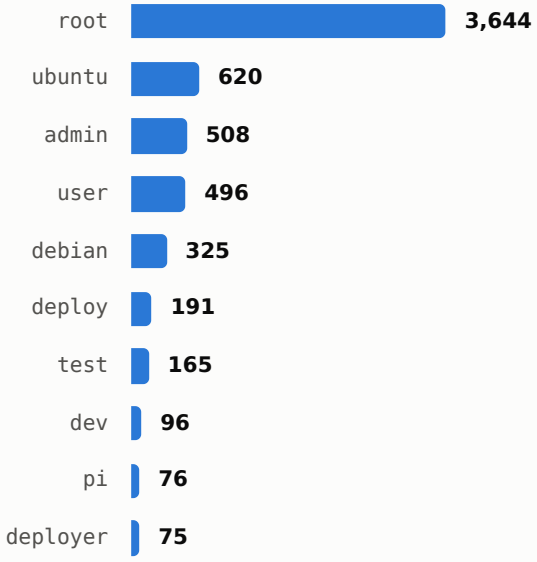
On RDP, it's even more concentrated: a **single source IP** (UK-hosted) accounts for **50%** of all RDP traffic on its own, and the top 3 IPs cover **79%**. RDP scanning in this window was dominated by a handful of determined actors rather than broad, diffuse noise.

What attackers type when they think no one's watching

Across the SSH honeypot, attackers tried **4,061 distinct username and password combinations** over **11,245 login attempts** (not every connection progressed to authentication): roughly 2.8 tries per unique pair, consistent with automated dictionary attacks rather than manual guessing.

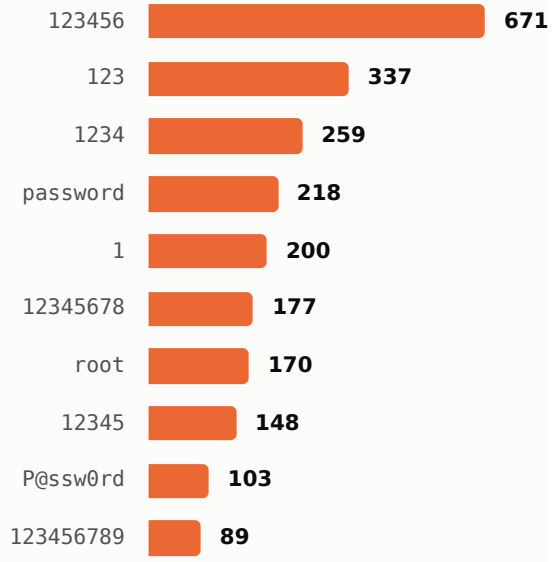
Top 10 usernames tried (SSH)

by total login attempts, all passwords combined



Top 10 passwords tried (SSH)

by total login attempts, all usernames combined



Nearly 1 in 3 SSH login attempts (32.4%) used the username `root`, by far the single most-targeted account on any exposed Linux box.

UNEXPECTED FIND: THE DICTIONARIES HAVE KEPT UP WITH 2026

Buried in the username list: `claude` (68 attempts, ranked #11 overall), `openclaw` (48, #21), and `codex` (15). All names associated with AI coding agents and AI-assisted dev tooling. Generic brute-force wordlists are evidently being refreshed to guess at accounts an "AI dev box" might plausibly have, right alongside classics like `ubuntu` and `admin`.

RDP: usernames without passwords

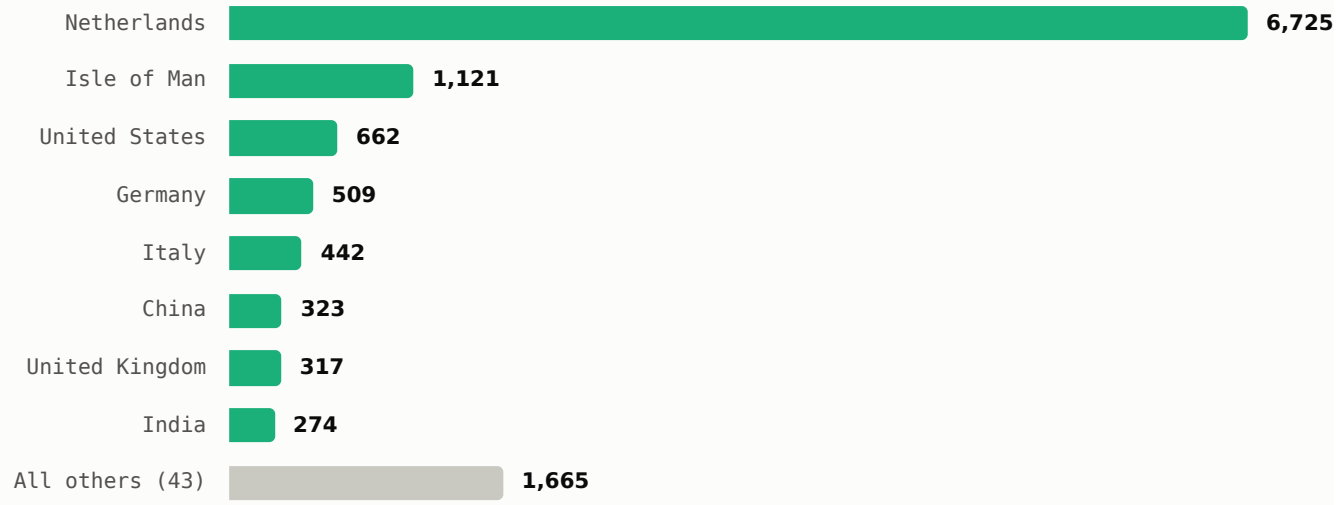
RDP's NLA/CredSSP handshake never sends a readable password. Trapster captured **37 distinct usernames** across **807 login-stage events** (each of the 612 network-level connections can generate multiple authentication events), but the large majority are single pre-TLS username probes with no follow-through (top guesses: `workgroup`, `win-rdp01`, `test`, `hello`). Only **one source IP** completed full NLA negotiation and ran an actual brute-force: submitting **administrator** against 14 distinct guessed passwords, recovered by cracking the captured NetNTLMv2 hashes. Everything else in the RDP data is reconnaissance, not a real login attempt.

A global footprint, but a concentrated core

Source IPs geolocated to **51 countries**, but the raw country ranking is dominated by a small number of hosting providers running many scanner instances from adjacent IP blocks, not by broad, organic geographic spread.

Connection attempts by country of origin

combined SSH + RDP, top 8 countries + all others (aggregated by IP geolocation)



THE NETHERLANDS NUMBER IS TWO HOSTING ACCOUNTS, NOT A COUNTRY TREND

98% of the Netherlands total (6,618 of 6,725 attempts) comes from just **two ISPs** running clusters of IPs in the same block: ten co-located addresses from one provider and three from another, each individually making several hundred attempts. Geolocation tells you where the server rack is, not who's operating it. Treat country-level rankings as an infrastructure map, not an attacker demographic.

A quieter signal: legitimate internet-wide scanners

Some of the traffic isn't attack traffic at all. IP ranges belonging to **Censys** and **Modat** (both known internet-mapping and attack-surface-scanning services) appear in the logs, and one SSH connection carried the client string `MGLNDD_<ip>_22`, a signature associated with commercial internet-scanning platforms. This is a useful reminder when reading any honeypot's totals: a meaningful slice of "attacker" traffic is research and attack-surface-mapping scanners cataloguing the internet, indistinguishable at the network layer from hostile recon.

A note on what's *not* published here

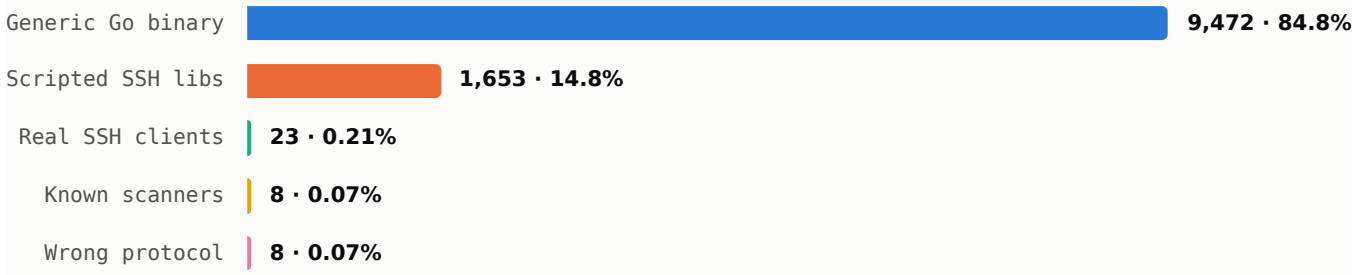
Individual source IP addresses are deliberately omitted from this report, consistent with how the underlying project handles this data (raw attacker IPs and credentials are excluded from version control by design). Everything above is aggregated to the country and ISP level, which preserves the pattern without publishing a list of specific hosts.

Almost none of this is a human at a keyboard

Cowrie logs each SSH client's raw identification banner and a HASSH fingerprint derived from its key-exchange proposal: together, a good proxy for "what software is on the other end of this connection."

SSH client tooling, by category (11,164 of 11,426 connections with a parseable client banner)

grouped from raw client version banners captured by Cowrie



Generic Go binary (84.8%): connections announcing themselves simply as `SSH-2.0-Go`, built with Go's standard SSH library and nothing else identifying. This is the signature of custom-compiled scanning and credential-stuffing tools, likely botnet agents, and by far the dominant source of traffic. At least four distinct HASSH key-exchange fingerprints account for 97% of this traffic, meaning it's several different tools or campaigns, not one.

Scripted SSH libraries (14.8%): almost entirely one version (`libssh_0.9.6`), the library behind countless small brute-force scripts distributed online. Its near-total dominance within this slice suggests one popular script or tutorial being run at scale.

Real interactive clients (0.21%): `paramiko` , `PuTTY` , `russh` , a couple of `OpenSSH` versions. The kind of tooling an actual person or a legitimate automation script would use. Vanishingly rare against this backdrop.

Known scanners (0.07%): recognizable research and recon tools, including `ZGrab` , `Nmap` 's SSH algorithm enumerator, `Fingerprintx` , and one MGLNDD-branded probe.

Wrong protocol (0.07%): a handful of connections sent a raw HTTP `GET /` request, or raw TLS `ClientHello` bytes, at port 22. Scanners blasting every protocol at every port without checking what's listening first.

THE HEADLINE

Over **99.9%** of fingerprinted SSH traffic came from generic, non-interactive tooling. In 24 hours, essentially zero traffic looked like "a person opened a terminal and typed `ssh` ." Everything hitting this box was automated: a fair proxy for what any newly provisioned, internet-facing SSH server should expect from minute one.

What this changes about how I think about exposure

1. **"Nobody will find my small server" is false, and fast.**

   RDP was probed 17 seconds after the VM came online, SSH within six minutes. Any public IP is discovered by continuous internet-wide scanning on a timescale of minutes, not days. Obscurity through a low profile buys you nothing.

2. **The volume gap between SSH and RDP is real and large.**

   Port 22 saw about 19 times the traffic of port 3389 in the same window. If you can only harden or closely monitor one exposed service, SSH is the busier target.

3. **Default and weak credentials remain the entire attack.**

   `root` / `123456`-class guesses dominate. There's no sophistication here to defend against: disabling password auth in favor of keys neutralizes nearly all of this traffic in one config change.

4. **Brute-force wordlists are quietly evolving with the industry.**

   Seeing `claude` (68 attempts), `openclaw` (48), and `codex` (15) as guessed usernames suggests dictionaries are being refreshed for accounts an "AI-assisted dev box" plausibly has. Worth remembering if you name service accounts after the tools running on them.

5. **Traffic is concentrated, not diffuse.**

   A handful of source IPs and hosting providers account for the majority of attempts on both ports. Rate-limiting or blocking a small number of persistent, repeat offenders would meaningfully cut the noise a real server sees.

6. **This is almost entirely machine-on-machine.**

   Over 99.9% of fingerprinted SSH clients were generic automated tooling. The mental model of "an attacker" here is closer to "one of a few dozen running bots" than "a person deciding to target you."

Limitations & where to find everything

Limitations

- A single 24-hour window on one cloud provider, one region, and one IP. Traffic volume and mix will vary with time, provider, and IP reputation and history.
- Deny-all authentication was a deliberate choice to maximize distinct credentials captured. It means **no post-login behavior** (commands run, payloads dropped) was observed. That's a natural next study, run as a separate, clearly-scoped instance.
- Country and ISP attribution is IP geolocation, which reflects hosting infrastructure, not necessarily the operator's actual location.
- RDP's NLA/CredSSP design means most "password" data captured is either absent (pre-auth username only) or an offline-crackable hash, not a verified plaintext credential.

Code & full methodology

The full source is public: provisioning scripts, honeypot configuration, and the analysis pipeline used to produce every number in this report.

github.com/tomershapira103/Honeypot